



ECOLE MAROCAINE DES
SCIENCES DE L'INGENIEUR

Membre de
HONORIS UNITED UNIVERSITIES

EMSI Casablanca

Filière Informatique — 4^{ème} IAD

Année universitaire 2025–2026

PROJET DE FIN DE MODULE

Deep Learning

*Conception, implementation, comparaison
et analyse critique de modèles de Deep Learning
pour données tabulaires, images et séquences*

Realiser par : Safaa Hassani

Encadrant : Mr. Batal

Framework : PyTorch 2.10

Environnement : Google Colab — GPU Tesla T4

Résumé

Ce rapport présente les travaux réalisés dans le cadre du projet de fin de module **Deep Learning** à l'**EMSI Casablanca** (filière 4IIR-IAD, 2025–2026).

L'objectif central est d'étudier, d'implémenter et de comparer les trois grandes familles d'architectures neuronales profondes :

- **Partie I** — Perceptron Multicouche (MLP) appliqué à la détection du cancer du sein (*Breast Cancer Wisconsin*), avec comparaison des méthodes d'initialisation, régularisation (Dropout, Weight Decay), Early Stopping et comparaison aux modèles classiques.
- **Partie II** — Réseau de Neurones Convolutionnel (CNN, architecture LeNet) pour la classification de chiffres manuscrits (MNIST), avec implémentation manuelle des opérations convolutionnelles et comparaison CNN vs MLP.
- **Partie III** — Modèles récurrents (RNN, LSTM, GRU) et système Seq2Seq encodeur-décodeur pour la traduction automatique Anglais→Espagnol, avec gradient clipping, teacher forcing, beam search et évaluation BLEU.

Tous les modèles ont été développés sous **PyTorch** et entraînés sur GPU **Tesla T4** via Google Colab.

Mots-clés : Deep Learning, MLP, CNN, LeNet, RNN, LSTM, GRU, Seq2Seq, Breast Cancer Wisconsin, MNIST, PyTorch, Régularisation, Traduction automatique.

Table des matières

Résumé	1
Introduction Générale	5
1 Partie I — Perceptron Multicouche (MLP)	6
1.1 Présentation du problème	6
1.2 Dataset : Breast Cancer Wisconsin	6
1.2.1 Répartition des classes	7
1.3 Analyse Exploratoire des Données	7
1.4 Prétraitement et Normalisation	7
1.4.1 Pourquoi normaliser ?	7
1.4.2 Prévention du <i>data leakage</i>	7
1.4.3 Séparation Train / Validation / Test	8
1.5 Architecture du MLP	8
1.5.1 Qu'est-ce qu'un MLP ?	8
1.5.2 Implémentation avec <code>nn.Sequential</code>	9
1.5.3 Implémentation via une classe <code>nn.Module</code>	9
1.5.4 Architecture finale : $30 \rightarrow 64 \rightarrow \text{ReLU} \rightarrow \text{Dropout}(0.3) \rightarrow 2$	9
1.6 Étude des Paramètres du Réseau	10
1.7 Comparaison des Stratégies d'Initialisation	10
1.8 Régularisation : Dropout et Weight Decay	11
1.8.1 Dropout	11
1.8.2 Weight Decay (Régularisation L2)	11
1.9 Early Stopping	11
1.10 Entraînement et Exécution sur CPU/GPU	12
1.10.1 Utilisation du GPU	12
1.11 Sauvegarde et Rechargement du Modèle	12
1.12 Métriques d'Évaluation et Résultats	13
1.12.1 Pourquoi plusieurs métriques ?	13
1.12.2 Résultats sur le jeu de test	14
1.12.3 Comparaison avec des modèles classiques de Machine Learning	14
1.12.4 Analyse et discussion des résultats	14
2 Partie II — Réseaux de Neurones Convolutionnels (CNN)	15
2.1 Présentation du problème	15
2.2 Dataset : MNIST	15

2.3	Fondements des Opérations Convolutionnelles	15
2.3.1	La corrélation croisée 2D : l'opération de base.....	15
2.3.2	Padding, Stride et Taille de sortie.....	16
2.3.3	Max Pooling et Average Pooling.....	16
2.3.4	Convolution 1×1	17
2.4	Architecture LeNet	17
2.4.1	Propagation des dimensions à travers le réseau.....	18
2.4.2	Nombre de paramètres.....	18
2.5	Entraînement.....	18
2.6	Résultats et Analyse.....	19
2.6.1	Évolution des performances pendant l'entraînement.....	19
2.6.2	Visualisation des feature maps.....	19
2.6.3	Comparaison CNN vs MLP sur MNIST	19
2.6.4	Analyse et discussion.....	19
3	Partie III — RNN, LSTM, GRU et Seq2Seq	21
3.1	Présentation du problème	21
3.2	Corpus anglais-espagnol	21
3.2.1	Génération et prétraitement	21
3.3	Modèle de Langage et Perplexité.....	22
3.4	Architectures Récurrentes.....	22
3.4.1	RNN Simple.....	22
3.4.2	LSTM (Long Short-Term Memory).....	22
3.4.3	GRU (Gated Recurrent Unit).....	23
3.5	BPTT et Gradient Clipping.....	23
3.6	Comparaison RNN / LSTM / GRU	24
3.7	Système Seq2Seq Encodeur-Décodeur.....	24
3.7.1	Teacher Forcing	25
3.8	Stratégies de Décodage.....	25
3.8.1	Décodage glouton.....	25
3.8.2	Beam Search ($k = 3$)	25
3.9	Évaluation : Score BLEU.....	26
3.10	Résultats et Analyse.....	26
3.10.1	Analyse et discussion des résultats.....	26
	Conclusion Générale	27

Liste des tableaux

1	Vue d'ensemble des architectures étudiées	5
1.1	Caractéristiques du dataset Breast Cancer Wisconsin	6
1.2	Répartition des classes — Breast Cancer Wisconsin	7
1.3	Rôle et taille de chaque ensemble de données	8
1.4	Détail des couches et paramètres du MLP	9
1.5	Comparaison des méthodes d'initialisation testées.....	10
1.6	Hyperparamètres d'entraînement — MLP Breast Cancer Wisconsin .	12
1.7	Métriques d'évaluation et leur interprétation clinique.....	13
1.8	Performances finales — MLP Breast Cancer Wisconsin (jeu de test).....	14
1.9	MLP vs modèles classiques — Breast Cancer Wisconsin.....	14
2.1	Caractéristiques du dataset MNIST	15
2.2	Paramètres d'une couche convolutionnelle et leur effet.....	16
2.3	Évolution des dimensions image à travers LeNet (entrée : $1 \times 28 \times 28$) .	18
2.4	Décompte des paramètres de LeNet (total : 61 706).....	18
2.5	Hyperparamètres d'entraînement — LeNet MNIST.....	18
2.6	Progression de la perte et de l'accuracy — LeNet sur MNIST.....	19
2.7	Comparaison MLP vs LeNet (CNN) — MNIST, 10 époques	19
3.1	Exemples de paires du corpus anglais → espagnol.....	21
3.2	Statistiques du corpus	22
3.3	Les trois portes du LSTM et leur rôle.....	23
3.4	Comparaison synthétique des architectures récurrentes.....	23
3.5	Résultats comparatifs — RNN vs LSTM vs GRU.....	24
3.6	Hyperparamètres du système Seq2Seq.....	25
3.7	Performances du système Seq2Seq — corpus EN→ES	26
3.8	Bilan synthétique des architectures étudiées.....	27
3.9	Performances obtenues sur les trois parties du projet.....	27

Introduction Générale

Le **Deep Learning** (apprentissage profond) est une branche de l'intelligence artificielle qui utilise des réseaux de neurones à plusieurs couches pour apprendre automatiquement des représentations utiles à partir de données brutes. Contrairement au Machine Learning classique où l'ingénieur choisit manuellement les caractéristiques, le réseau les apprend lui-même.

Aujourd'hui, le Deep Learning est au cœur des avancées en reconnaissance d'images, traduction automatique, détection médicale et génération de texte.

TABLE 1 – Vue d'ensemble des architectures étudiées

Partie	Architecture	Dataset	Type de données
I	MLP (Perceptron Multicouche)	Breast Cancer Wisconsin	Tabulaires
II	CNN (LeNet)	MNIST	Images
III	RNN / LSTM / GRU / Seq2Seq	Corpus EN→ES	Séquences textuelles

Chaque architecture est naturellement adaptée à un type de données : le MLP traite les données tabulaires, le CNN exploite la structure spatiale des images, et les modèles récurrents capturent les dépendances dans les séquences. L'ensemble des expériences a été réalisé sur **Google Colab** avec un GPU **Tesla T4** et PyTorch 2.10.0.

1. Partie I — Perceptron Multicouche (MLP)

1.1 Présentation du problème

La **détection du cancer du sein** consiste à classer une tumeur comme *bénigne* (sans danger) ou *maligne* (cancéreuse) à partir de mesures réalisées sur des images de cellules. C'est un problème de **classification binaire** sur des données médicales tabulaires.

Ce problème est particulièrement important : une erreur de diagnostic peut avoir des conséquences graves. Détecter un cancer malin comme bénin retarde le traitement ; l'inverse génère une anxiété et des examens inutiles. Le choix des métriques d'évaluation devra en tenir compte.

1.2 Dataset : Breast Cancer Wisconsin

TABLE 1.1 – Caractéristiques du dataset Breast Cancer Wisconsin

Propriété	Valeur
Nombre d'observations	569 patientes
Nombre de variables	30 mesures numériques
Valeurs manquantes	Aucune
Tâche	Classification binaire (Bénin / Malin)
Source	UCI Machine Learning Repository

Les 30 variables décrivent des propriétés géométriques des noyaux cellulaires détectés sur une image : rayon, texture, périmètre, aire, lissé, compacité, concavité, symétrie et dimension fractale. Chacune est calculée de trois façons (valeur moyenne, écart-type, valeur maximale), ce qui donne $10 \times 3 = 30$ variables.

1.2.1 Répartition des classes

TABLE 1.2 – Répartition des classes — Breast Cancer Wisconsin

Classe	Signification	Effectif	Proportion
0	Malin (Malignant) — tumeur cancéreuse	212	37.3%
1	Bénin (Benign) — tumeur non cancéreuse	357	62.7%
Total		569	100%

Le jeu de données est légèrement déséquilibré (37 %/63 %), ce qui est représentatif d'une situation clinique réelle. Ce déséquilibre reste modéré et ne nécessite pas de rééchantillonnage particulier.

1.3 Analyse Exploratoire des Données

Avant d'entraîner un modèle, il est essentiel de comprendre les données. L'analyse exploratoire a révélé plusieurs points importants :

- **Plages de valeurs très différentes** : certaines variables ont des valeurs autour de 0.01 (lissé) tandis que d'autres dépassent 2 500 (aire). Sans normalisation, les variables à grande échelle domineraient l'apprentissage.
- **Corrélations entre variables** : les mesures de rayon, périmètre et aire sont fortement corrélées entre elles (ce sont des propriétés géométriques liées). Certaines variables sont redondantes, mais le MLP peut gérer cette redondance sans prétraitement supplémentaire.
- **Pouvoir discriminant** : les variables de rayon, périmètre et compacité montrent des distributions bien séparées entre Bénin et Malin, ce qui indique que la tâche est réalisable avec un modèle relativement simple.

1.4 Prétraitement et Normalisation

1.4.1 Pourquoi normaliser ?

Les 30 variables ont des échelles très différentes. Si on entraîne directement le réseau sur ces valeurs brutes, les poids associés aux grandes variables auraient beaucoup plus d'influence que les autres, ce qui biaiserait l'apprentissage.

La **normalisation Z-score** recentre chaque variable autour de 0 avec un écart-type de 1, mettant toutes les variables sur un pied d'égalité :

$$x_{\text{normalisé}} = \frac{x - \text{moyenne}}{\text{écart-type}}$$

1.4.2 Prévention du *data leakage*

Qu'est-ce que le data leakage ?

Un piège classique consiste à calculer la moyenne et l'écart-type sur **toutes** les données, y compris le test. Le modèle aurait alors indirectement accès à des statistiques du test pendant l'entraînement, ce qui donnerait des performances artificiellement gonflées.

La bonne pratique : calculer ces statistiques **uniquement sur le train** et les appliquer ensuite sans modification sur la validation et le test.

```

1 scaler = StandardScaler ()
2 X_train = scaler.fit_transform (X_train )   # calcule ET applique
        sur train
3 X_val    = scaler.transform (X_val)          # applique les memes
        parametres
4 X_test   = scaler.transform (X_test)         # idem pour le test

```

Listing 1.1 – Normalisation correcte sans data leakage

1.4.3 Séparation Train / Validation / Test

TABLE 1.3 – Rôle et taille de chaque ensemble de données

Ensemble	Exemples	Proportion	Rôle
Entraînement	398	70%	Ajuster les poids du réseau
Validation	85	15%	Surveiller le surapprentissage, choisir les hyperparamètres
Test	86	15%	Évaluation finale et honnête du modèle
Total	569	100%	—

Pourquoi trois ensembles ? Si on utilisait le test pour choisir les hyperparamètres, on optimiserait implicitement pour le test et les performances ne seraient plus représentatives de nouvelles données inconnues. La validation joue le rôle de garde-fou lors du développement.

1.5 Architecture du MLP

1.5.1 Qu'est-ce qu'un MLP ?

Un **Perceptron Multicouche** est un réseau de neurones entièrement connecté. Chaque neurone d'une couche est relié à tous les neurones de la couche suivante. Le signal se propage de l'entrée vers la sortie (propagation avant), puis les erreurs se propagent en sens inverse pour ajuster les poids (rétropropagation).

1.5.2 Implémentation avec nn.Sequential

PyTorch permet de définir un MLP de manière concise avec nn.Sequential, qui empile les couches dans l'ordre :

```

1 model = nn.Sequential(
2     nn.Linear(30, 64),      # 30 entrees -> 64 neurones caches
3     nn.ReLU (),            # activation non-lineaire
4     nn.Dropout (0.3) ,      # regularisation : desactive 30% des
5                             # neurones
6     nn.Linear(64, 2)        # 64 entrees -> 2 sorties (Benin /
7                             # Malin)
8 )

```

Listing 1.2 – MLP via nn.Sequential

1.5.3 Implémentation via une classe nn.Module

Une approche plus flexible consiste à créer une classe qui hérite de nn.Module. Cela permet de contrôler précisément la propagation avant :

```

1 class MLP(nn.Module):
2     def __init__(self, input_size=30, hidden=64, output_size=2):
3         super(MLP, self).__init__()
4         self.fc1 = nn.Linear(input_size, hidden)
5         self.fc2 = nn.Linear(hidden, output_size)
6         self.relu = nn.ReLU()
7         self.dropout = nn.Dropout(0.3)
8
9     def forward(self, x):
10        x = self.relu(self.fc1(x))    # couche 1 + activation
11        # ReLU
12        x = self.dropout(x)           # regularisation
13        x = self.fc2(x)               # couche de sortie
14        return x

```

Listing 1.3 – MLP personnalisé via nn.Module

1.5.4 Architecture finale : 30 → 64 → ReLU → Dropout(0.3) → 2

TABLE 1.4 – Détail des couches et paramètres du MLP

#	Couche	Entrée	Sortie	Paramètres	Rôle
1	Linear	30	64	$30 \times 64 + 64 = 1\,984$	Transformation linéaire
–	ReLU	64	64	0	Introduire la non-linéarité
–	Dropout	64	64	0	Régularisation (30 % des neurones)
2	Linear	64	2	$64 \times 2 + 2 = 130$	Score pour chaque classe
Total paramètres				2 114	–

Pourquoi ReLU ? L'activation ReLU laisse passer les valeurs positives telles quelles et met les valeurs négatives à zéro. Sans activation, plusieurs couches linéaires équivaleraient à une seule, et le réseau ne pourrait pas apprendre des relations non-linéaires.

1.6 Étude des Paramètres du Réseau

PyTorch offre deux outils pour inspecter les paramètres d'un réseau :

```
1 for name, param in model.named_parameters():
2     print(f'{name} | forme : {param.shape} | nb : {param.numel()}')
3 # fc1 . weight | forme : [64, 30] | nb : 1920
4 # fc1 . bias   | forme : [64]      | nb :    64
5 # fc2 . weight | forme : [2, 64]   | nb :   128
6 # fc2 . bias   | forme : [2]       | nb :    2
```

Listing 1.4 – Inspection des paramètres du réseau

Le `state_dict` est le dictionnaire qui contient tous les poids et biais du réseau. C'est lui qu'on sauvegarde sur le disque et qu'on recharge pour réutiliser un modèle entraîné.

1.7 Comparaison des Stratégies d'Initialisation

Pourquoi l'initialisation est-elle importante ?

Au démarrage de l'entraînement, les poids sont initialisés à de petites valeurs aléatoires. Un mauvais choix peut provoquer deux problèmes :

- **Gradient qui s'évanouit** : poids trop petits, le signal s'atténue en traversant les couches et le réseau n'apprend plus.
- **Gradient qui explose** : poids trop grands, le signal explose et l'entraînement diverge.

TABLE 1.5 – Comparaison des méthodes d'initialisation testées

Méthode	Principe	Observation
Gaussienne	Poids tirés d'une loi normale très petite ($\sigma = 0.01$)	Convergence lente : signal trop faible au début
Constante	Tous les poids fixés à 0.1	A éviter : tous les neurones calculent la même chose, symétrie non brisée
Xavier uniform	Poids tirés dans un intervalle calculé en fonction du nombre d'entrées/sorties de la couche	Retenue : amplitude stable d'une couche à l'autre, convergence plus rapide

L'initialisation **Xavier** est la plus appropriée pour un réseau avec des activations

ReLU car elle adapte l'amplitude des poids à la taille de la couche, évitant à la fois l'évanouissement et l'explosion du signal.

1.8 Régularisation : Dropout et Weight Decay

Le surapprentissage (*overfitting*) survient quand le réseau mémorise les données d'entraînement au lieu d'en extraire les tendances générales. Deux techniques ont été mises en place pour l'éviter :

1.8.1 Dropout

Le Dropout désactive aléatoirement 30 % des neurones à chaque passage lors de l'entraînement. En empêchant le réseau de s'appuyer sur des neurones spécifiques, le Dropout force chaque neurone à apprendre quelque chose d'utile indépendamment des autres. Cela rend le réseau plus robuste et plus général.

Important : le Dropout est désactivé automatiquement lors de l'évaluation (`model.eval()`), car on veut utiliser le réseau complet pour la prédiction.

1.8.2 Weight Decay (Régularisation L2)

La régularisation L2 pénalise les poids trop grands en les poussant vers zéro pendant l'entraînement. Des poids trop grands indiquent souvent que le réseau s'est trop spécialisé sur des exemples particuliers. Dans PyTorch, cette régularisation est intégrée dans l'optimiseur :

```
1 optimizer = torch.optim.Adam(model.parameters(),
2                               lr=0.001,
3                               weight_decay=1e-4)
4 # weight_decay=1e-4 penalise les poids de grande valeur
```

Listing 1.5 – Adam avec Weight Decay (régularisation L2)

1.9 Early Stopping

Principe de l'Early Stopping

On surveille la perte sur la validation après chaque époque. Si elle ne s'améliore plus pendant un certain nombre d'époques consécutives (la *patience*), on arrête l'entraînement et on garde le meilleur modèle vu jusqu'ici.

C'est particulièrement utile sur de petits datasets comme Breast Cancer Wisconsin (398 exemples d'entraînement) où le surapprentissage peut survenir rapidement.

```
1 best_val_loss = float('inf')
2 patience = 10 # on attend 10 epoques sans amelioration
3 counter = 0
4
5 for epoch in range(num_epochs):
6     # ... phase d'entrainement ...
```

```

7
8     if val_loss < best_val_loss:                # amelioration
9         detectee
10         best_val_loss = val_loss
11         counter = 0
12         torch.save(model.state_dict(), "best_model.pth")
13     else:
14         counter += 1
15         if counter >= patience :
16             print (f"Arret a l'epoque {epoch+1}")
17             break

```

Listing 1.6 – Implémentation de l'Early Stopping

1.10 Entraînement et Exécution sur CPU/GPU

1.10.1 Utilisation du GPU

Les calculs d'un réseau de neurones sont essentiellement des multiplications de matrices. Les GPU sont conçus pour effectuer ces opérations en parallèle, ce qui les rend beaucoup plus rapides que les CPU. Dans PyTorch, il suffit de déplacer le modèle et les données sur le GPU avec `.to(device)` :

```

1 device = torch.device("cuda" if torch.cuda.is_available() else
2                       "cpu")
3 model = model.to(device)
4 X_train_t = X_train_t.to(device)
5 y_train_t = y_train_t.to(device)
6 # Resultat : Device utilise = cuda (Tesla T4)

```

Listing 1.7 – Déplacement du modèle et des données sur GPU

TABLE 1.6 – Hyperparamètres d'entraînement — MLP Breast Cancer Wisconsin

Hyperparamètre	Valeur	Justification
Optimiseur	Adam	Adaptatif, bon sur petits datasets
Taux d'apprentissage	0.001	Valeur standard
Weight Decay	10^{-4}	Légère pénalisation des grands poids
Dropout	0.3	30 % des neurones désactivés
Fonction de perte	CrossEntropyLoss	Adaptée à la classification
Initialisation	Xavier uniforme	Meilleure convergence observée
Patience (early stop)	10 époques	Arrêt si pas d'amélioration
Device	GPU Tesla T4	Calcul accéléré

1.11 Sauvegarde et Rechargement du Modèle

Après chaque époque, on vérifie si le modèle actuel est meilleur que le précédent. Si oui, on sauvegarde ses poids. À la fin de l'entraînement, on recharge ces meilleurs poids

pour l'évaluation finale. Cette approche garantit qu'on évalue le modèle au moment où il avait la meilleure généralisation.

```

1 torch.save(model.state_dict(), "best_model.pth") #
   sauvegarde
2
3 model.load_state_dict(torch.load("best_model.pth")) #
   rechargement
4 model.eval() # mode evaluation : desactive le Dropout

```

Listing 1.8 – Sauvegarde et rechargement du meilleur modèle

1.12 Métriques d'Évaluation et Résultats

1.12.1 Pourquoi plusieurs métriques ?

L'accuracy seule peut être trompeuse. Avec 62.7 % de cas bénins, un modèle qui prédit “bénin” pour tous les patients aurait 62.7 % d'accuracy sans jamais rien avoir appris ! On utilise donc des métriques complémentaires :

TABLE 1.7 – Métriques d'évaluation et leur interprétation clinique

Métrique	Ce qu'elle mesure	Importance clinique
Accuracy	Proportion globale de bonnes prédictions	Indicateur général
Precision	Parmi les cas prédits Malins, combien sont vraiment Malins	Évite les fausses alarmes
Recall	Parmi les vrais Malins, combien ont été détectés	Prioritaire : ne pas rater un cancer
F1-Score	Moyenne harmonique entre Precision et Recall	Équilibre global
Matrice de confusion	Tableau complet des vrais/faux positifs et négatifs	Vue détaillée des erreurs

En contexte médical, le **Recall** est la métrique la plus importante : manquer un cancer malin (faux négatif) est beaucoup plus grave que signaler à tort un cas bénin (faux positif).

1.12.2 Résultats sur le jeu de test

TABLE 1.8 – Performances finales — MLP Breast Cancer Wisconsin (jeu de test)

Métrique	Valeur	Interprétation
Accuracy	> 95%	Très bonne classification globale
Precision	> 0.94	Peu de fausses alarmes
Recall	> 0.94	Peu de cancers manqués
F1-Score	> 0.94	Bon équilibre général

1.12.3 Comparaison avec des modèles classiques de Machine Learning

TABLE 1.9 – MLP vs modèles classiques — Breast Cancer Wisconsin

Modèle	Description	Accuracy	F1-Score
Régression Logistique	Modèle linéaire, interprétable	$\approx 95\%$	≈ 0.94
Forêt Aléatoire	Ensemble d'arbres, très robuste	$\approx 96\%$	≈ 0.95
MLP (notre modèle)	Réseau neuronal avec régularisation	> 95%	> 0.94

1.12.4 Analyse et discussion des résultats

- **Le MLP est compétitif** avec les modèles classiques sur ce dataset. Les trois approches atteignent environ 95–96 %, ce qui suggère que la frontière de décision entre Bénin et Malin est quasi-linéaire dans cet espace de variables normalisées.
- **La Régression Logistique** est un excellent modèle de référence : simple, interprétable, rapide. Ses performances proches du MLP indiquent que les relations entre variables et cible sont relativement simples.
- **La Forêt Aléatoire** surpasse légèrement le MLP sur ce petit dataset (569 exemples). C'est un résultat attendu : les méthodes ensemblistes sont reconnues pour leur robustesse sur de petits volumes de données.
- **L'apport du Dropout et du Weight Decay** se voit dans la faible divergence entre les courbes de perte train et validation : le modèle généralise bien sans mémoriser les données d'entraînement.
- **L'Early Stopping** s'est avéré essentiel avec seulement 398 exemples d'entraînement : le réseau peut rapidement sur-ajuster sans ce mécanisme de contrôle.

2. Partie II — Réseaux de Neurones Convolutionnels (CNN)

2.1 Présentation du problème

La **reconnaissance de chiffres manuscrits** consiste à identifier quel chiffre (de 0 à 9) est représenté dans une image en niveaux de gris. C'est une **classification à 10 classes** sur des données d'image.

Ce problème illustre la limite du MLP sur les images : si on aplatissait une image 28×28 en un vecteur de 784 valeurs, on perdrait toute l'information spatiale (quels pixels sont voisins). Le CNN est conçu précisément pour exploiter cette structure spatiale.

2.2 Dataset : MNIST

TABLE 2.1 – Caractéristiques du dataset MNIST

Propriété	Valeur
Images d'entraînement	60 000
Images de test	10 000
Résolution	28×28 pixels
Type	Niveaux de gris (1 canal)
Nombre de classes	10 (chiffres 0 à 9)

Prétraitement : chaque pixel est normalisé pour avoir une valeur entre -1 et 1 . Les images sont chargées en mini-batches de 64 pour l'entraînement.

2.3 Fondements des Opérations Convolutionnelles

2.3.1 La corrélation croisée 2D : l'opération de base

Comment fonctionne une convolution ?

Imaginez un filtre (ou noyau) de petite taille, par exemple 5×5 pixels. On le fait

glisser sur toute l'image, et à chaque position on calcule la somme des produits entre le filtre et la région de l'image qu'il recouvre. Le résultat est une nouvelle image appelée **carte de caractéristiques** (feature map).

Un filtre appris pour détecter les bords verticaux activera fortement sur les zones de l'image avec un bord vertical, et faiblement ailleurs. En empilant plusieurs filtres, le CNN construit une représentation riche de l'image.

Cette opération a été implémentée manuellement en NumPy et comparée au résultat de PyTorch :

```

1 def correlation_croisee_2d(image, filtre):
2     h_in, w_in = image.shape
3     h_k, w_k = filtre.shape
4     h_out, w_out = h_in - h_k + 1, w_in - w_k + 1
5     sortie = np.zeros((h_out, w_out))
6     for i in range(h_out):
7         for j in range(w_out):
8             sortie[i, j] = np.sum(image[i:i+h_k, j:j+w_k] *
9             filtre)
10    return sortie
11 print ("Identiques :", np.allclose(result_manuel,
12    # --> Identiques : True

```

Listing 2.1 – Implémentation manuelle de la corrélation croisée 2D

2.3.2 Padding, Stride et Taille de sortie

TABLE 2.2 – Paramètres d'une couche convolutionnelle et leur effet

Paramètre	Définition	Effet
Stride (s)	Pas de déplacement du filtre	Un stride de 2 divise la taille de sortie par 2
Padding (p)	Ajout de zéros autour de l'image	Contrôle la taille de sortie ; avec $p = (k - 1)/2$, la taille est conservée

La taille de la carte de sortie se calcule : $H_{\text{sortie}} = \lfloor (H_{\text{entrée}} + 2p - k)/s \rfloor + 1$.

2.3.3 Max Pooling et Average Pooling

Après une convolution, on applique une opération de **pooling** pour réduire la taille spatiale tout en conservant l'information essentielle :

- **Max Pooling** : retient la valeur maximale dans chaque région. Capte la présence d'un motif, indépendamment de sa position exacte. C'est l'opération la plus utilisée car elle est robuste aux petits décalages.

- **Average Pooling** : calcule la moyenne dans chaque région. Donne une information plus lissée, utilisée dans l’architecture LeNet.

Les deux ont été implémentés manuellement en NumPy et validés contre PyTorch.

2.3.4 Convolution 1×1

Une convolution avec un filtre 1×1 ne regarde qu’un seul pixel à la fois. Son rôle est de **combiner les canaux** entre eux sans toucher aux dimensions spatiales : elle permet de réduire ou d’augmenter le nombre de canaux.

2.4 Architecture LeNet

L’architecture **LeNet-5**, proposée par Yann LeCun en 1998, est l’une des premières architectures CNN efficaces. Elle alterne des couches de convolution (extraction de features) et de pooling (réduction de dimension), puis termine par des couches entièrement connectées pour la classification.

```

1  class LeNet( nn. Module ):
2      def __init__( self ):
3          super (LeNet, self). __init__ ()
4          # Bloc 1 : Convolution + Pooling
5          self. conv1 = nn. Conv2 d(1, 6, kernel_size=5, padding =2)
6          # 28x28 -> 28x28
7          self. pool1 = nn. Avg Pool2 d( kernel_size=2, stride=2)
8          # 28x28 -> 14x14
9          # Bloc 2 : Convolution + Pooling
10         self. conv2 = nn. Conv2 d(6, 16, kernel_size=5)
11         # 14x14 -> 10x10
12         self. pool2 = nn. Avg Pool2 d( kernel_size=2, stride=2)
13         # 10x10 -> 5x5
14         # Couches entierement connectees
15         self. fc1 = nn. Linear (16*5*5 , 120)
16         self. fc2 = nn. Linear (120 , 84)
17         self. fc3 = nn. Linear (84 , 10)      # 10 classes
18
19     def forward (self, x):
20         x = torch .sigmoid (self. conv1 (x)); x = self. pool1 (x)
21         x = torch .sigmoid (self. conv2 (x)); x = self. pool2 (x)
22         x = x.view (-1, 16*5*5)              # aplatir
23         x = torch .sigmoid (self. fc1 (x))
24         x = torch .sigmoid (self. fc2 (x))
25         return self. fc3 (x)

```

Listing 2.2 – Implémentation de LeNet pour MNIST

2.4.1 Propagation des dimensions à travers le réseau

TABLE 2.3 – Évolution des dimensions image à travers LeNet (entrée : $1 \times 28 \times 28$)

Couche	Opération	Sortie (C×H×W)	Taille totale
Entrée	Image originale	$1 \times 28 \times 28$	784
Conv1	6 filtres 5×5 , $p = 2$	$6 \times 28 \times 28$	4 704
AvgPool1	2×2 , stride 2	$6 \times 14 \times 14$	1 176
Conv2	16 filtres 5×5	$16 \times 10 \times 10$	1 600
AvgPool2	2×2 , stride 2	$16 \times 5 \times 5$	400
Flatten	Aplatissage	400	400
FC1	Linéaire	120	120
FC2	Linéaire	84	84
FC3	Linéaire (sortie)	10	10

2.4.2 Nombre de paramètres

TABLE 2.4 – Décompte des paramètres de LeNet (total : 61 706)

Couche	Calcul	Paramètres
Conv1	6 filtres $\times$ (1 canal $\times$ 25 + 1 biais)	156
Conv2	16 filtres $\times$ (6 canaux $\times$ 25 + 1 biais)	2 416
FC1	$400 \times 120 + 120$	48 120
FC2	$120 \times 84 + 84$	10 164
FC3	$84 \times 10 + 10$	850
Total		61 706

2.5 Entraînement

TABLE 2.5 – Hyperparamètres d’entraînement — LeNet MNIST

Hyperparamètre	Valeur	Justification
Nombre d’époques	10	Convergence rapide sur MNIST
Taille de batch	64	Bon équilibre vitesse/stabilité
Optimiseur	Adam	Standard et efficace
Taux d’apprentissage	0.001	Valeur classique
Fonction de perte	CrossEntropyLoss	Adaptée aux 10 classes
Activation (conv)	Sigmoid	Conforme à LeNet original

2.6 Résultats et Analyse

2.6.1 Évolution des performances pendant l'entraînement

TABLE 2.6 – Progression de la perte et de l'accuracy — LeNet sur MNIST

Époque	Train Loss	Test Accuracy	Observation
1	1.0198	92.16%	Convergence très rapide dès la 1 ^{re} époque
2	0.2190	95.51%	Gain de +3 points en une seule époque
3	0.1390	97.07%	Passage au-dessus de 97 %
5	0.0842	97.63%	Plateau progressif
8	0.0553	98.35%	Performances très élevées
10	0.0464	98.41%	Résultat final

Le réseau atteint déjà 92 % d'accuracy dès la première époque. Cela montre que les couches convolutionnelles apprennent très rapidement des caractéristiques discriminantes (contours, formes des chiffres).

2.6.2 Visualisation des feature maps

Après la première couche convolutionnelle, on obtient 6 cartes de caractéristiques. Chacune correspond à un filtre différent et met en évidence un type de motif distinct (bords horizontaux, verticaux, textures). C'est la preuve que le CNN apprend automatiquement des détecteurs de caractéristiques, sans qu'on les ait définis manuellement.

2.6.3 Comparaison CNN vs MLP sur MNIST

Pour illustrer l'avantage du CNN sur les images, un MLP simple ($784 \rightarrow 256 \rightarrow 128 \rightarrow 10$) a été entraîné sur les mêmes images :

TABLE 2.7 – Comparaison MLP vs LeNet (CNN) — MNIST, 10 époques

Modèle	Paramètres	Accuracy (test)	Remarque
MLP simple	$\approx 235\,000$	$\approx 97\%$	Plus de paramètres, moins bon
LeNet (CNN)	61 706	98.41%	4× moins de paramètres, meilleur

2.6.4 Analyse et discussion

- **Efficacité du partage de poids** : LeNet utilise 4 fois moins de paramètres que le MLP simple tout en obtenant de meilleures performances. Ce gain vient du **partage des poids** : le même filtre est appliqué à toute l'image, ce qui est logique car un bord vertical peut apparaître n'importe où.
- **Invariance par translation** : grâce au pooling, le CNN reconnaît un chiffre même s'il est légèrement décalé. Un MLP classique doit mémoriser séparément chaque variation de position.

- **Hiérarchie de features** : les premières couches détectent des motifs simples (bords), les couches suivantes les combinent en formes plus complexes (courbes, segments des chiffres).
- **Limite de l'activation Sigmoid** : l'architecture LeNet originale utilise Sigmoid, qui peut saturer et ralentir l'apprentissage. L'activation ReLU, plus moderne, accélérerait probablement la convergence.

3. Partie III — RNN, LSTM, GRU et Seq2Seq

3.1 Présentation du problème

Cette partie porte sur la **traduction automatique** de l’anglais vers l’espagnol. La difficulté ne réside pas seulement dans la traduction des mots : l’espagnol suit des règles grammaticales différentes de l’anglais.

- En anglais, l’adjectif précède le nom : “the *red* car”
- En espagnol, l’adjectif suit le nom **et s’accorde en genre** : “el coche *rojo*” (masculin) ou “la casa *roja*” (féminin)

Le modèle doit donc apprendre simultanément le vocabulaire bilingue, l’ordre des mots et l’accord grammatical.

3.2 Corpus anglais-espagnol

3.2.1 Génération et prétraitement

Un corpus contrôlé a été généré à partir d’un lexique de 12 noms et 10 adjectifs, combinant 4 patrons de phrases. Le dédoublonnage produit $\approx 6\,000$ paires.

TABLE 3.1 – Exemples de paires du corpus anglais $\rightarrow$ espagnol

Anglais	Espagnol	Phénomène illustré
the red car	el coche rojo	Réordonnancement + accord masc.
a small house	una casa pequeña	Accord féminin
i see the black bird	veo el pajar negro	Réordonnancement
the yellow flower	la flor amarilla	Accord féminin

Les étapes de prétraitement sont : tokenisation, construction du vocabulaire (avec tokens spéciaux SOS/EOS/PAD), encodage en indices, padding à 10 tokens, et chargement en mini-batches de 64.

TABLE 3.2 – Statistiques du corpus

Propriété	Valeur
Paires uniques totales	$\approx 6\,000$
Entraînement	85% ($\approx 5\,100$)
Test	15% (≈ 900)
Vocabulaire anglais	35 mots + SOS/EOS/PAD
Vocabulaire espagnol	43 mots + SOS/EOS/PAD

3.3 Modèle de Langage et Perplexité

Qu'est-ce qu'un modèle de langage ?

Un modèle de langage prédit le mot suivant dans une phrase en fonction de tous les mots précédents. La **perplexité** mesure à quel point le modèle est “surpris” par les vraies données : une perplexité faible signifie que le modèle prédit bien le mot suivant. Un modèle qui devine au hasard parmi 43 mots (vocabulaire espagnol) aurait une perplexité de 43. Un bon modèle vise une perplexité bien inférieure.

3.4 Architectures Récurrentes

3.4.1 RNN Simple

Comment fonctionne un RNN ?

À chaque pas de temps t , le RNN reçoit le mot actuel et l'état caché précédent (qui résume tout ce qui a été vu avant), et produit un nouvel état caché. Cet état caché est une sorte de **mémoire** qui se transmet d'un mot à l'autre.

Le problème : cette mémoire s'efface rapidement pour les longues séquences. C'est le problème du **gradient qui s'évanouit**.

3.4.2 LSTM (Long Short-Term Memory)

Le LSTM résout le problème du RNN simple en ajoutant une **mémoire à long terme** contrôlée par trois *portes* :

TABLE 3.3 – Les trois portes du LSTM et leur rôle

Porte	Rôle	Analogie
Porte d’oubli	Décide quelle information oublier de la mémoire précédente	“Dois-je oublier le genre du sujet actuel ?”
Porte d’entrée	Décide quelle nouvelle information stocker en mémoire	“Dois-je mémoriser ce nouveau sujet ?”
Porte de sortie	Décide quelle partie de la mémoire exposer en sortie	“Quelle information est utile pour prédire maintenant ?”

Grâce à ces portes, le LSTM peut **sélectivement mémoriser ou oublier** des informations sur de longues séquences, résolvant ainsi le problème de l’évanouissement du gradient.

3.4.3 GRU (Gated Recurrent Unit)

Le GRU est une version simplifiée du LSTM avec seulement deux portes. Il est moins coûteux en calcul que le LSTM pour des performances souvent similaires sur des séquences courtes.

TABLE 3.4 – Comparaison synthétique des architectures récurrentes

Architecture	Portes	Complexité	Mémoire LT	Stabilité
RNN simple	0	Faible	Mauvaise	Instable
LSTM	3	Élevée	Excellente	Bonne
GRU	2	Moyenne	Bonne	Bonne

3.5 BPTT et Gradient Clipping

BPTT et gradient clipping

Pour entraîner un RNN, on propage les erreurs en arrière à travers chaque pas de temps (BPTT). Le problème est que ce gradient peut devenir très grand (explosion) ou très petit (évanouissement) sur de longues séquences.

Le **gradient clipping** règle le problème de l’explosion : si la norme du gradient dépasse un seuil (ici 1.0), on le réduit proportionnellement. Cela stabilise l’entraînement sans bloquer l’apprentissage.

```

1 loss.backward()
2 nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
3 optimizer.step()
```

Listing 3.1 – Gradient clipping dans la boucle d’entraînement

L'expérience montre que **sans clipping**, la perte du RNN simple oscille ou diverge dès les premières époques. **Avec clipping**, la courbe de perte descend régulièrement vers un minimum stable.

3.6 Comparaison RNN / LSTM / GRU

TABLE 3.5 – Résultats comparatifs — RNN vs LSTM vs GRU

Architecture	Perplexité	Stabilité	Temps entraî.	Remarque
RNN simple	Élevée	Instable sans clipping	Rapide	Problème gradient lo
LSTM	Basse	Bonne	Plus long	Meilleure mémoire lo
GRU	Basse	Bonne	Moyen	Bon compromis vitesse/

Sur des séquences courtes (max 10 tokens), les trois architectures donnent des résultats comparables après convergence. L'avantage du LSTM et du GRU serait plus visible sur des séquences longues où la mémoire à long terme devient essentielle.

3.7 Système Seq2Seq Encodeur–Décodeur

Principe du système Seq2Seq

Le modèle est composé de deux parties :

- L'**encodeur** lit la phrase source (anglais) mot par mot et compresse toute l'information en un **vecteur de contexte**.
- Le **décodeur** prend ce vecteur et génère la phrase cible (espagnol) mot par mot.

C'est comparable à un interprète qui lit et comprend une phrase entière avant de commencer à la traduire.

```

1 class Encoder(nn.Module):
2     def __init__(self, vocab_size, embed_size=128, hidden_size
3         =256):
4         super ().__init__ ()
5         self.embedding = nn.Embedding (vocab_size, embed_size,
6             padding_idx = PAD_token )
7         self.gru = nn.GRU (embed_size, hidden_size, batch_first
8             =True)
9
10        def forward (self, x):
11            outputs, hidden = self.gru (self.embedding (x))
12            return outputs, hidden    # hidden = vecteur de
13                                     contexte
14
15 class Decoder(nn.Module):
16     def __init__(self, vocab_size, embed_size=128, hidden_size
17         =256):

```

```

14     super ().__init__ ()
15     self.embedding = nn.Embedding (vocab_size , embed_size ,
16                                   padding_idx = PAD_token )
17     self.gru = nn.GRU (embed_size , hidden_size , batch_first
18 =True )
19     self.fc = nn.Linear( hidden_size , vocab_size )
20
21 def forward (self, x, hidden):
22     output , hidden = self.gru (self.embedding (x), hidden )
23     return self.fc( output), hidden

```

Listing 3.2 – Encodeur et Décodeur GRU du système Seq2Seq

3.7.1 Teacher Forcing

Qu'est-ce que le Teacher Forcing ?

Pendant l'entraînement, si le décodeur se trompe sur un mot, l'erreur risque de se propager sur tous les mots suivants. Pour éviter cela, on utilise le **teacher forcing** : avec une probabilité de 50 %, on donne au décodeur le *vrai* mot cible plutôt que sa propre prédiction.

C'est comme un professeur qui corrige un élève en direct : si l'élève dit le mauvais mot, on lui donne immédiatement le bon pour qu'il continue correctement.

TABLE 3.6 – Hyperparamètres du système Seq2Seq

Hyperparamètre	Valeur	Rôle
Nombre d'époques	40	Convergence sur ce corpus
Taille de batch	64	Standard
Taille d'embedding	128	Représentation dense des mots
Taille état caché	256	Capacité mémoire du modèle
Teacher forcing ratio	0.5	50 % supervision directe
Gradient clipping	1.0	Stabilité de l'entraînement

3.8 Stratégies de Décodage

3.8.1 Décodage glouton

À chaque pas de temps, on choisit simplement **le mot le plus probable**. C'est rapide mais peut manquer de meilleures solutions globales.

3.8.2 Beam Search ($k = 3$)

Au lieu de garder un seul candidat, le beam search maintient les **3 meilleures hypothèses partielles** à chaque pas. À la fin, on retient la séquence complète avec le meilleur score. Il donne de meilleurs résultats que le décodage glouton lorsque le modèle est incertain.

3.9 Évaluation : Score BLEU

Qu'est-ce que le score BLEU ?

Le BLEU mesure à quel point la traduction produite ressemble à la référence, en comptant les mots (1-grammes) et groupes de mots (2-grammes) en commun. Un BLEU de 1.0 signifie que la traduction est exactement identique à la référence.

Le BLEU est la métrique standard en traduction automatique car l'Accuracy seule est trop stricte : plusieurs traductions correctes peuvent exister pour une même phrase.

3.10 Résultats et Analyse

TABLE 3.7 – Performances du système Seq2Seq — corpus EN→ES

Stratégie	BLEU moyen	Traductions exactes	Remarque
Greedy (glouton)	≈ 1.0	$\approx 100\%$	Rapide
Beam Search ($k = 3$)	≈ 1.0	$\approx 100\%$	Coïncide avec greedy ici

3.10.1 Analyse et discussion des résultats

- **Correspondance exacte de $\approx 100\%$** sur le jeu de test, même sur des combinaisons nom-adjectif jamais vues pendant l'entraînement. Cela prouve que le modèle a appris la **règle compositionnelle** (lexique + ordre des mots + accord en genre) et ne s'est pas contenté de mémoriser les exemples.
- **Régularité du corpus** : ce résultat exceptionnel est dû à la simplicité du corpus contrôlé. Sur un vrai corpus (Wikipedia, livres), les performances chuteraient car le vocabulaire serait de dizaines de milliers de mots et les constructions grammaticales beaucoup plus variées.
- **Limite du goulot d'étranglement** : toute l'information de la phrase source est compressée en un seul vecteur de taille fixe (256). Pour des phrases longues, cette compression perd de l'information. C'est précisément ce problème que le mécanisme d'**attention** résout.
- **Beam search vs greedy** : les deux méthodes donnent le même résultat car le modèle est très confiant sur chaque mot. Dans un contexte d'ambiguïté, le beam search aurait un avantage réel.

Conclusion Générale

Ce projet a permis d'implémenter, d'analyser et de comparer les trois grandes familles d'architectures du Deep Learning en les appliquant à des problèmes réels.

Bilan comparatif des architectures

TABLE 3.8 – Bilan synthétique des architectures étudiées

Architecture	Données idéales	Points forts	Limitations
MLP	Données tabulaires	Simple, peu de paramètres, efficace avec bonne normalisation	Ne capture pas la structure spatiale ni temporelle
CNN (LeNet)	Images, 2D, grilles	Partage de poids, invariance aux translations, hiérarchie de features	Moins adapté aux séquences
RNN / LSTM / GRU / Seq2Seq	Séquences	Capture les dépendances temporelles	Goulot d'étranglement pour les longues séquences

Récapitulatif des performances

TABLE 3.9 – Performances obtenues sur les trois parties du projet

Partie	Modèle	Dataset	Performance
I	MLP (2 114 params)	Breast Cancer Wisc. (569)	Accuracy > 95%, F1 > 0.94
II	LeNet (61 706 params)	MNIST (70 000)	Accuracy = 98.41%
III	Seq2Seq GRU	EN→ES (≈ 6 000)	BLEU ≈ 1.0, exact ≈ 100%

Leçons apprises

1. **L'architecture doit correspondre à la nature des données** : un MLP sur des données tabulaires, un CNN sur des images, des modèles récurrents sur des séquences. La mauvaise architecture conduit à de mauvaises performances même avec beaucoup de paramètres.
2. **Le prétraitement est aussi important que le modèle** : sans normalisation correcte (MLP), sans tokenisation (Seq2Seq) ou sans DataLoader structuré (CNN), même le meilleur modèle ne peut pas converger.
3. **La régularisation prévient le surapprentissage** : Dropout, Weight Decay, gradient clipping et Early Stopping sont des outils complémentaires qui permettent au modèle de généraliser plutôt que mémoriser.
4. **Comparer avec des baselines est essentiel** : la comparaison avec la Régression Logistique et la Forêt Aléatoire (Partie I) et entre MLP et CNN (Partie II) permet de mesurer la vraie valeur ajoutée de chaque architecture.
5. **Comprendre les outils en profondeur** : implémenter manuellement les opérations convolutionnelles et les valider contre PyTorch a permis de comprendre réellement le fonctionnement interne des CNN.

Perspectives d'amélioration

- **Mécanisme d'attention** pour le Seq2Seq : permettrait au décodeur de regarder directement toutes les positions de la phrase source, éliminant le goulot d'étranglement.
- **Architecture Transformer** : le standard actuel en traitement du langage, basé entièrement sur l'attention et les connexions résiduelles.
- **Transfer Learning** : utiliser des modèles pré-entraînés (ResNet pour les images, BERT pour le texte) affinés sur notre tâche permettrait d'excellents résultats même avec peu de données.
- **Activation ReLU dans LeNet** : remplacer Sigmoid par ReLU dans les couches convolutionnelles accélérerait probablement la convergence.

Projet réalisé dans le cadre du module Deep Learning.

EMSI Casablanca — Filière Informatique 4IIR-IAD — Année universitaire 2025–2026